

How Figure 1 parallel training maps to SeqTrainer

A paper-to-code walkthrough of the neural-memory update

Focus: Figure 1 and equations 11-18, not an end-to-end DNA model.



Bottom line

The recurrence and its gradients are implemented; Figure 1s parallel reformulation is documented as the next optimization step, not silently substituted for the reference.

1. What Figure 1 is accelerating

Before Figure 1: a neural MLP stores associations by updating its own fast weights.

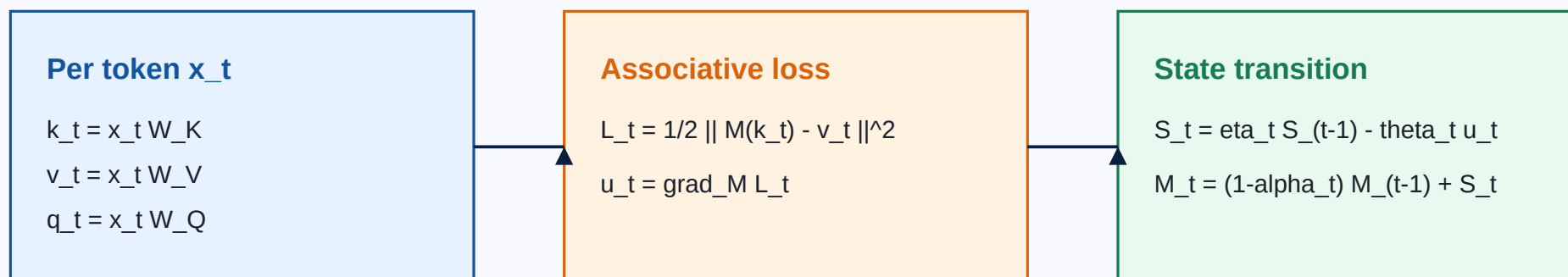


Figure 1 asks: how can the same update be evaluated in chunk-sized matrix operations?

Paper Figure 1 caption: training neural memory in parallel using matmuls; Section 3.2 develops equations 16-18.

2. Equation 16: unrolling forgetting inside a chunk

Paper idea: replace a token-by-token weight update with a weighted sum of chunk gradients.

Paper equation 16, simplified notation

$$M_t = \beta_t M_0 - \sum_i \theta_i (\beta_t / \beta_i) \text{grad } L(M_{tprime}; x_i)$$

$\beta_i = \prod_{j \leq i} (1 - \alpha_j)$, and M_{tprime} is the state at the start of the chunk.

Why it helps

All token gradients in a chunk can be combined as tensors.

What is preserved

Token-dependent alpha and theta still weight each contribution.

SeqTrainer choice

Keep the recurrence explicit for C=32 correctness first.

3. Equation 17: the linear-memory matmul example

For a linear memory W and one chunk X , the paper batches many outer-product gradients.

Paper equation 17

$$\sum_i \text{weighted grad } L(W_0; x_i) = \text{Theta}_b B_b (W_0 X - X) X^T$$

Diagonal gate matrices weight rows; matrix products replace a Python loop over the b tokens.

Closest SeqTrainer primitives

```
key_projection(x), value_projection(x)  
functional_call(memory_mlp, fast_weights)
```

Important difference

SeqTrainer supports an MLP, so gradients come from
`torch.autograd.grad(..., create_graph=True)`

The paper says the MLP case is analogous; Stage A does not claim the closed-form linear matmul is implemented for MLP weights.

4. Equation 18: momentum becomes an associative scan

The paper separates the momentum recurrence from weight accumulation.

$$S_t = \eta_t S_{(t-1)} - \theta_t u_t$$

u_t is the associative-loss gradient.

Parallel-paper route

Compute all u_t together, then use an associative scan over η_t .

SeqTrainer reference route

```
for position in range(32):  
    candidate = update_one(candidate, ...)  
Exact order, simple to test, O(C) Python control flow.
```

Why keep it?

It is the oracle for a future scan.
The tests expose a detach, reordered update, or wrong gate application immediately.

5. SeqTrainer: exact recurrence-to-code map

FunctionalNeuralMemory

```
associative_loss: 0.5 * (M(k) - v)^2  
surprise_gradient: autograd.grad(...)  
momentum_update: eta*S - theta*grad  
forgetting_update: (1-alpha)*M + S
```

update_segment

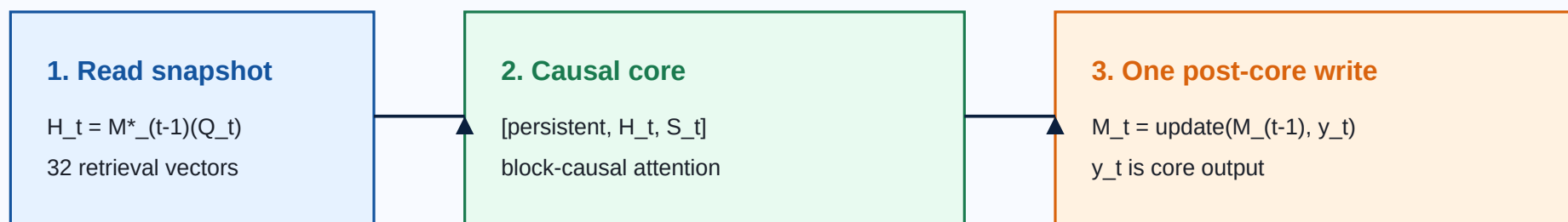
```
keys, values = projections(segment)  
gates = gates(segment) # all 32 at once  
candidate = state  
for position in range(32): update_one(...)
```

Autograd contract

Fast weights and surprise are replacement tensors, never a mutable slot buffer. `create_graph=True` preserves gradients through all 32 updates.

Code: `src/seqtrainer/torch/titans_paper_mac/memory.py`; tests: `tests/test_titans_paper_mac_memory.py`

6. MAC integration: Figure 1 memory meets paper equations 21-25



Important Stage A correction

PaperMACBlock now calls `read_segment`, then `integrate`, then `update_segment(state, sequence)`. It does not write raw input embeddings.

Code: `mac.py`; proof: `test_block_reads_a_full_prewrite_segment_then_commits_one_state`.

7. Side-by-side: paper intent versus current implementation

Mechanism	Paper / Figure 1	SeqTrainer Stage A
Associative loss	K/V projections + squared reconstruction	Functional MLP loss; exact gradient
Forgetting + momentum	alpha, eta, theta recurrence	Token gates; sequential reference
Chunked matmuls	Eq. 16-17 tensorize gradients	Not yet implemented
Associative scan	Eq. 18 parallel momentum	Not yet implemented
MAC write	Write attention/core y_t	Implemented after causal integration

Why the gap is healthy

The reference path is a test oracle. A future optimized scan/convolution must agree with it on values and gradients before replacing nothing.

Do not read performance claims into Figure 1 coverage: this repository currently prioritizes paper-traceable correctness.

8. How to reproduce and extend the comparison

Reproduce the reference

```
uv sync --extra torch --extra dev
uv run pytest -q
uv run seqtrainer-titans-stage-a-benchmark
```

Read the audit

docs/titans_stage_a/fidelity_audit.md

Records what is faithful, controlled,
and deliberately deferred.

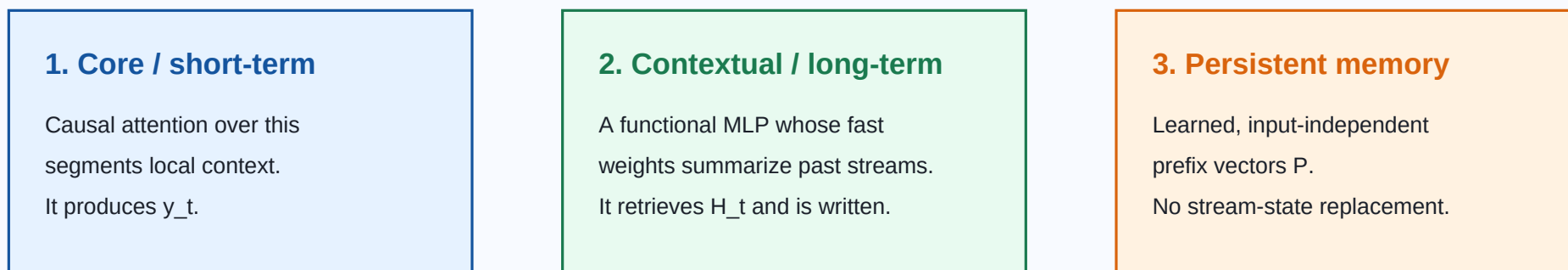
If implementing Figure 1 acceleration next

1. Keep the sequential C=32 path.
2. Add a scan/matmul implementation behind a separate API.
3. Compare states, surprise, outputs, and gradients against the reference under fixed seeds.
4. Keep stream isolation and the block-causal no-future-leakage tests unchanged.

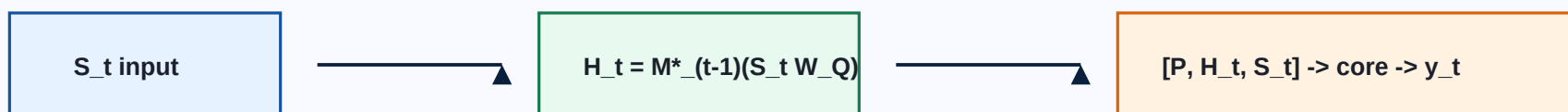
Source: paper Section 3.2 / Figure 1, and SeqTrainer commit 46dd015.

9. Figure 2: MAC combines three different kinds of memory

The paper's MAC design uses three branches with distinct lifetimes and update rules.



One segment, conceptually



Paper: Figure 2 and equations 19, 21-25. The arrows describe data flow, not three independent models.

10. Core: the attention branch chooses what to combine

Paper role: limited-window short-term processing. SeqTrainer role: one inspectable causal attention core.

Paper expression

$Z_t = [P, H_t, S_t]$

$Y_t = \text{Attention}(Z_t, \text{mask})$

The sequence portion becomes the core result.

PaperMACBlock.integrate

```
layout = cat((persistent, retrieval, segment))
attended = self.attention(layout, layout, layout, mask)
return output_norm(segment + attended[sequence])
```

Causality contract: the mask makes position i see P plus $H_{1:i}$ and $S_{1:i}$.

For safety, persistent-token queries see only P . This is a Stage A interpretation stricter than Figure 2's diagram.

The core is intentionally minimal: MultiheadAttention + residual + LayerNorm, not a full LM stack.

Code: `mac.py:147-169`; mask proof: `tests/test_titans_paper_mac_block.py:10-27`.

11. Contextual long-term memory: a stream-owned functional MLP

Long-term memory is not a bank of token slots; it is an MLP evaluated with per-stream fast weights.

Read: fixed $M_{(t-1)}$ snapshot

```
queries = query_projection(S_t)
H_t = functional_call(memory_mlp, fast_weights, queries)
```

All 32 retrievals come from the same state.

Write: replacement M_t

```
keys, values = projections(y_t)
for position in range(32): update_one(...)
```

The final candidate replaces stream fast weights.

Critical timing distinction

Reads use raw incoming S_t as queries; writes learn associations from post-core y_t . Therefore the core filters what reaches M_t .
The memory state is immutable-by-replacement and keyed by stream ID; it is not an `nn.Parameter` updated by the optimizer.

Code: `memory.py:121-129, 199-236, 257-266; mac.py:180-182.`

12. Persistent memory: trainable task prefix, not online state

Paper equation 19 prepends learned, input-independent memory to every context.

Implementation

```
self.persistent_tokens = nn.Parameter(empty(P, d))
normal_(persistent_tokens, mean=0, std=.02)
persistent = persistent_tokens.to(device, dtype)
```

How it participates

```
layout = cat((persistent, retrieval, segment))
```

The same P vectors prefix every 32-token segment.

They are not stored in PaperMACStreamState.

Training versus test-time behavior

During ordinary training, `loss.backward()` gives `persistent_tokens` an outer-loop gradient alongside core parameters.

At test time an application does not run an optimizer step, so P remains fixed; long-term fast weights still update per segment.

Code: `mac.py:107-120, 156-160`. Test: `persistent_tokens` receives a gradient in `test_titans_paper_mac_block.py:91-104`.

13. One training-time segment: ownership and gradient paths

PaperMACBlock.forward is a forward/update transition; an enclosing trainer owns the outer optimizer step.



Inner loop: online, stateful

M_t and surprise_t flow to the next segment for this stream. They are replacement tensors.

Outer loop: ordinary training

Loss through y_t and M_t reaches attention, P, MLP initialization, projections, and gates.

Evidence

The block test backpropagates a loss on sequence and fast weights, proving gradients reach segment input, persistent tokens, attention, and gates.

The minimal Stage A package does not define a production trainer; it correctly preserves the graph that such a trainer consumes.

14. Figure 2 to SeqTrainer: precise implementation status

Figure 2 branch	Paper responsibility	SeqTrainer Stage A implementation
Core	Short-term context processing	Causal MultiheadAttention + residual/norm
Contextual long-term memory	Memorize/retrieve historical abstractions	Functional MLP in per-stream fast state
Persistent memory	Input-independent task knowledge	nn.Parameter $P \times d$, prepended each segment
Read/write orchestration	Retrieve, attend, update from core output	read_segment -> integrate -> update_segment
Test-time update	Long-term learns; P is fixed	Fast state updates without optimizer

The important mental model

P is a learned model parameter; M_t is per-stream runtime state; the core is the differentiable function that joins P , H_t , and S_t .
The block never mutates $M_{(t-1)}$: it returns M_t . That separation makes streaming, causal tests, and meta-gradients inspectable.

Paper Figure 2 / Section 4.1; implementation: src/seqtrainer/torch/titans_paper_mac/mac.py and memory.py.